05. CDR — Concept Definition Report for the Planned Private Investment Decision Platform

Integrated concept-definition report covering product vision, market positioning, architecture, UX, validation, technology, and delivery

Purpose	Serve as the controlling concept report for the full planned platform
Primary use case	Private medium-term portfolio decision support for one expert owner-operator
System identity	Weight-centric decision-intelligence platform
Core architectural pattern	Selection -> Allocation -> Timing -> Portfolio-Level Risk Overlay

This Concept Definition Report consolidates the concept for the planned private investment platform into one integrated document that can guide product direction, architecture, UX, validation design, and phased implementation.

The product is designed for a single informed user and therefore can optimize for depth, rigor, and continuity rather than for mass-market simplification. That freedom should be used carefully: to increase decision quality, not to justify clutter or gratuitous complexity.

The system’s role is to combine text and news analysis, social-sentiment analysis, technical context, machine-learning signals, reinforcement-learning-inspired policy components, portfolio analytics, and operational truth into a coherent recommendation object that is understandable and auditable.

A key design insight from the prior analysis is that the true gap versus FinRL-X is not the number of models but the absence of a unified decision framework.

A second key insight is that the product should evolve from a multi-engine analytics dashboard into a weight-centric portfolio decision system with explicit Selection, Allocation, Timing, and Portfolio-Level Risk Overlay stages.

Target Architecture

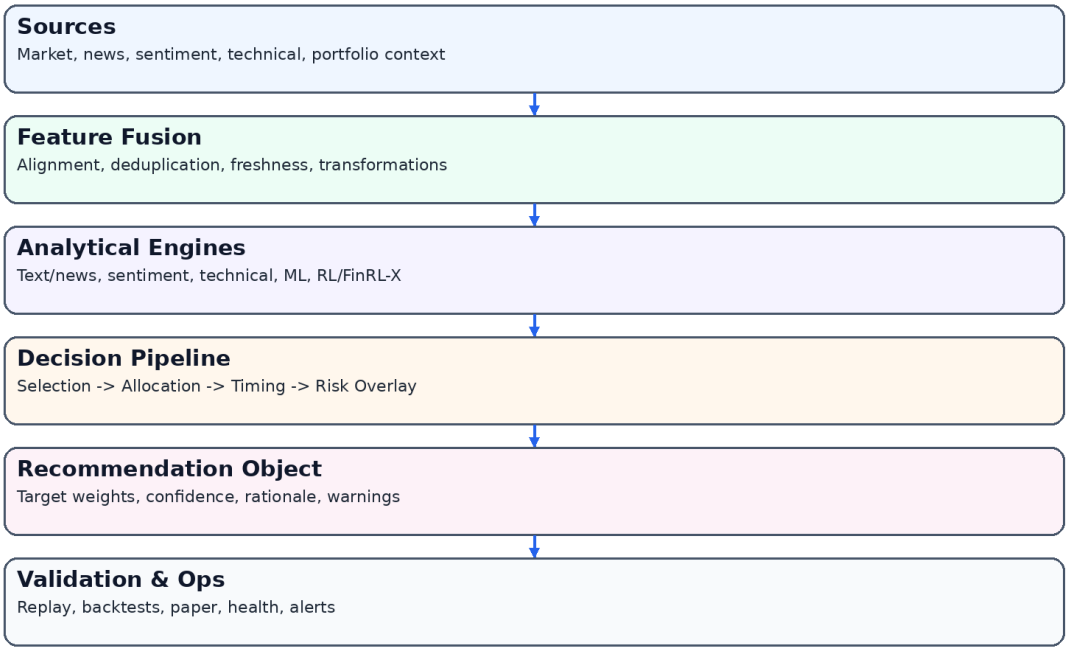

Figure 1. Integrated target architecture.

Market Landscape

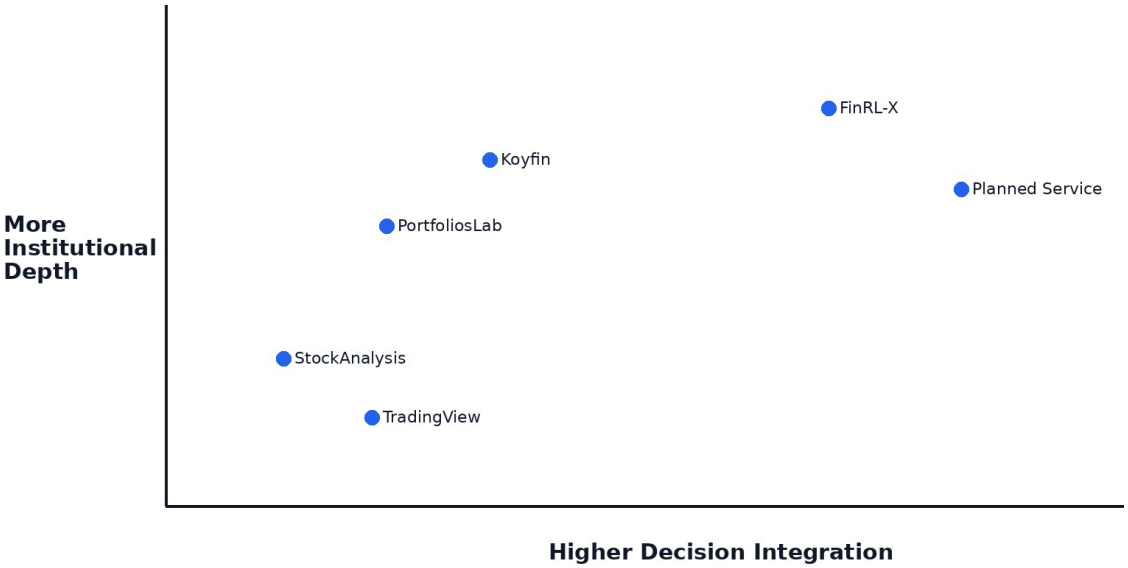

Figure 3. Market positioning.

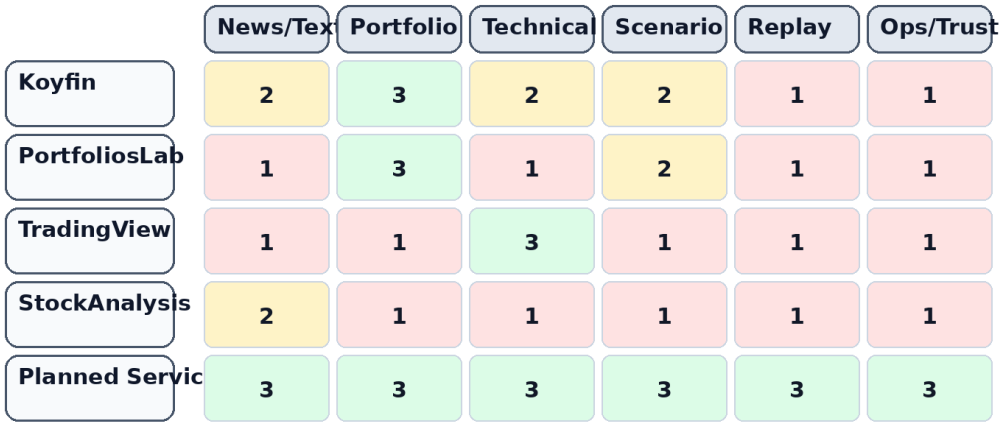

Figure 4. Competitive synthesis.

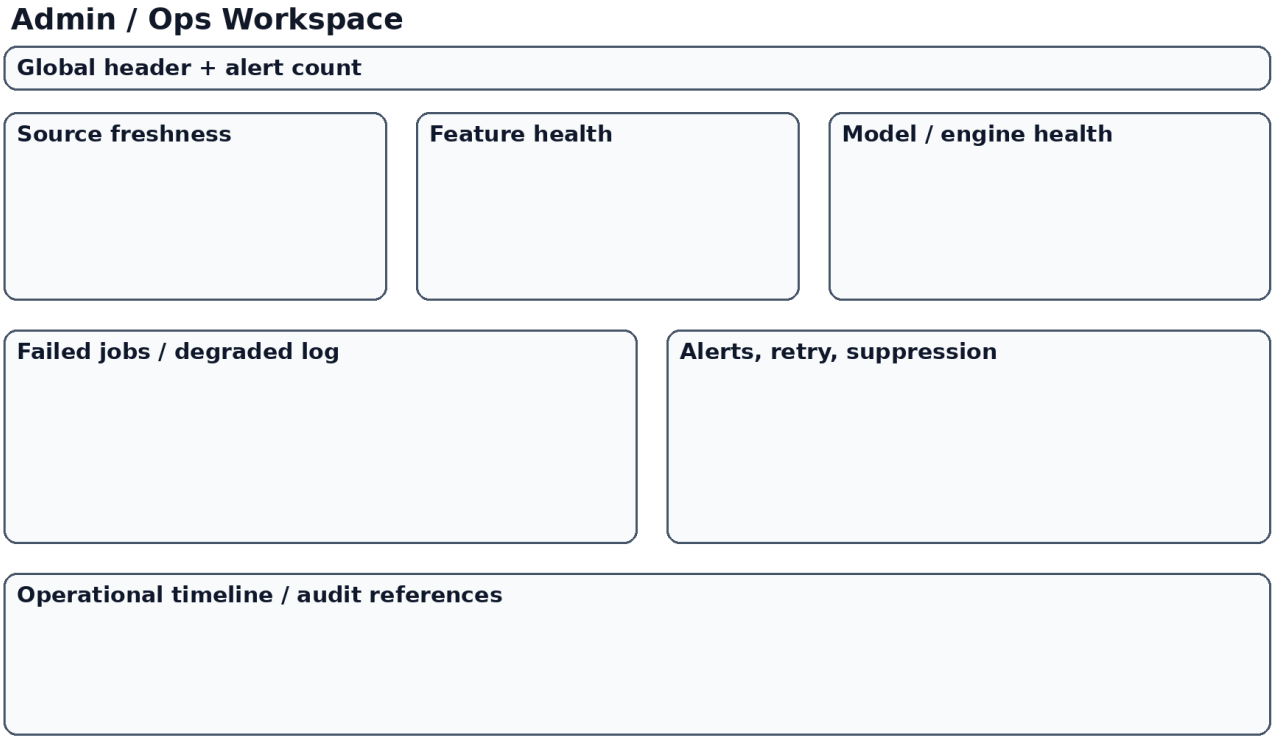

Figure 7. Admin / Ops concept.

Dimension	CDR recommendation
Product identity	Private decision-intelligence platform for medium-term portfolio decisions
Core object	Stable recommendation object with target weights, trust, warnings, and audit link
Policy stack	Selection, Allocation, Timing, Risk Overlay
Validation stack	Replay, backtests, paper portfolio, admin/ops visibility
Technology path	Web-first, mobile-strong, PWA-capable, app-ready
Delivery model	Phased build with evidence and review gates

1. Mission and Strategic Intent

The mission is to improve medium-term portfolio decisions by reducing fragmentation between signals, context, and portfolio consequences.

The strategic intent is to create a private cockpit for judgment, not a public trading product.

1. Mission and Strategic Intent must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

2. Product Thesis

The product thesis is that fragmented intelligence is the main problem. Valuable signals exist, but they are rarely converted into one coherent and auditable portfolio action.

The platform should therefore win by integration quality, trust, and decision clarity rather than by the raw number of widgets or feeds.

2. Product Thesis must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

3. User Context and Operating Model

The product is for one owner-operator, which means continuity, speed, depth, and control matter more than generic onboarding or social features.

This context supports a product that is more explicit and policy-oriented than mainstream retail apps.

3. User Context and Operating Model must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

4. Scope and Non-Goals

The scope includes recommendation generation, validation, replay, paper simulation, and operational truth.

The initial product is not a brokerage replacement and should not over-commit to live execution complexity.

4. Scope and Non-Goals must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

5. Market Opportunity and Positioning

The market is populated by research platforms, portfolio analytics tools, charting products, and AI overlays, but the planned product occupies a narrower and more coherent private niche.

Its value comes from turning diverse evidence into one portfolio decision while exposing trust and risk honestly.

5. Market Opportunity and Positioning must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

6. Architecture Overview

The system should move from sources to features to engines to policy layers to recommendation object to validation and ops.

The architecture should privilege stable contracts and replayable decisions.

6. Architecture Overview must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

7. Data and Feature Layer

The data layer should include market, text/news, sentiment, technical, macro, and portfolio state where relevant.

Freshness and alignment are first-class concerns because stale or mismatched data directly degrade trust.

7. Data and Feature Layer must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

8. Analytical Engines

Text/news analysis should capture catalysts, narrative shifts, and event relevance.

Social sentiment should add context but should be treated with explicit trust constraints.

Technical analysis should play a major role in timing and regime awareness.

ML and RL should be used where they truly improve ranking, allocation, or policy behavior.

8. Analytical Engines must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

9. Unified Decision Framework

The uploaded note correctly identified that the missing core is a decision framework rather than another model. This CDR adopts that conclusion fully.

The platform should therefore be defined by how it integrates stages, not by how many engines it contains.

9. Unified Decision Framework must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

10. Selection Layer

Selection defines the investable shortlist or candidate ranking based on universe rules and combined evidence.

It prevents silent inconsistency across engines and creates a stable decision boundary for downstream layers.

10. Selection Layer must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

11. Allocation Layer

Allocation converts shortlisted opportunities into proposed target weights or exposures.

The first release should allow simple allocators while preserving the contract needed for richer strategies later.

11. Allocation Layer must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

12. Timing Layer

Timing decides whether to delay, reduce, or confirm exposure changes under prevailing conditions.

For a weeks-to-months product, timing is a major quality layer rather than a high-frequency trading feature.

12. Timing Layer must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

13. Portfolio-Level Risk Overlay

Risk overlay is where the system enforces concentration, volatility, drawdown, and related portfolio-level constraints.

This layer should remain visible in both backend logic and frontend explanation.

13. Portfolio-Level Risk Overlay must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

14. Recommendation Object

The recommendation object should contain target weights, deltas, confidence decomposition, risk state, drivers, warnings, and an audit link.

Its stability across UI, replay, and validation surfaces is a core product requirement.

14. Recommendation Object must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

15. Trust Model

Trust should be decomposed into model confidence, data confidence, and operational confidence.

This keeps the product honest about where uncertainty actually originates.

15. Trust Model must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

16. UX and Screen System

The platform should be built around Overview, Dashboard, Engine Comparison, Risk, Replay, Backtests, Paper Portfolio, and Admin/Ops.

This screen system should mirror the decision lifecycle rather than backend ownership boundaries.

16. UX and Screen System must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

17. Dashboard Concept

The main dashboard is the primary working environment and must show recommendation, chart, risk, trust, and controls clearly.

It should not devolve into a dashboard of equal-weight cards.

17. Dashboard Concept must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into

one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

18. Replay and Validation

Replay, backtests, and paper portfolio should be treated as first-class validation surfaces.

They are how the system earns the right to be trusted over time.

18. Replay and Validation must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

19. Admin and Operational Honesty

Admin/Ops is where the system tells the truth about source freshness, feature health, model state, and publication condition.

This is a product-quality layer, not merely a technical support feature.

19. Admin and Operational Honesty must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

20. Technology Recommendation

The recommended path is web-first with a Python analytical backend and a React/Next.js frontend, optimized for responsive dashboards and PWA readiness.

A future iPhone path should be considered only after evidence justifies native investment.

20. Technology Recommendation must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

21. Delivery and Governance

Implementation should proceed in phases with evidence packs, acceptance gates, and explicit review points.

Schemas, warnings, degraded modes, fixtures, and ADRs should be treated as delivery assets.

21. Delivery and Governance must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

22. Risks and Mitigations

Major risks include false confidence, dashboard clutter, overbuilding, stale data, and weak schema discipline.

Mitigations include confidence decomposition, replayability, operational truth, and phased delivery.

22. Risks and Mitigations must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

23. Final Concept Recommendation

Proceed with a private decision platform built around a weight-centric recommendation object and a visible policy pipeline.

The strongest differentiator is not one model family, but the discipline with which diverse evidence becomes a coherent, auditable decision.

23. Final Concept Recommendation must be treated as a design and implementation constraint, not merely as a research observation. For this platform, decisions about interface hierarchy, data contracts, recommendation schema, and validation surfaces all flow from the same product thesis: the system exists to convert heterogeneous evidence into one coherent and auditable portfolio recommendation. That means every screen and every backend contract should reinforce the policy pipeline rather than dilute it. A consulting-grade interpretation also requires practical consequences. In this context, that means the section should produce explicit requirements for screen structure, trust semantics, validation approach, and delivery sequencing rather than ending at generic commentary.

Appendix A — Product Requirements Summary

Requirement area	Priority	Summary
Recommendation clarity	Critical	User must understand current recommendation in seconds
Trust visibility	Critical	Model/data/ops confidence must be separate
Risk visibility	Critical	Risk should be visible throughout key workflows
Replayability	High	Every recommendation should be replayable
Validation surfaces	High	Backtests and paper portfolio should align with recommendation schema

Mobile strategy	High	Mobile should optimize for triage and review

Appendix B — Entity Summary

Entity	Purpose
Source snapshot	Preserve raw inputs and freshness state
Feature set	Aligned transformed input context
Engine output	Structured upstream analytical result
Selection result	Shortlist or ranking stage output
Allocation proposal	Pre-risk portfolio proposal
Timing adjustment	Delay/reduce/confirm change state
Risk overlay result	Post-risk constrained recommendation state
Recommendation object	Published decision object
Replay run	Step-by-step audit package
Health snapshot	Operational state object

Appendix C — Delivery Phases

Phase	Primary scope	Exit condition
Phase 1	Contracts, entities, recommendation schema, baseline APIs	Stable foundation exists
Phase 2	Dashboard shell, trust, risk, controls, warnings	Correct hierarchy exists
Phase 3	Comparison, risk workspace, replay, backtests	Validation surfaces exist
Phase 4	Paper portfolio, admin/ops, degraded-mode logic	Operational truth exists
Phase 5	Hardening, accessibility, QA, consistency	Release readiness evidence exists

Integration of Uploaded Notes

The capability-gap note has been incorporated as a structural principle: the product should not be defined by the number of models but by the quality of its unified decision framework.

The architecture-update note has been incorporated as a concept-level reframing: the system should evolve into a modular portfolio decision platform centered on Selection, Allocation, Timing, and Portfolio-Level Risk Overlay.

References

This CDR also reflects current official positioning from AI4Finance around FinRL-X / FinRL-Trading, including its description as modular, reproducible, and production-ready, as well as the FinRL repository's direction that production-oriented development should use FinRL-X / FinRL-Trading.

Technology direction in this report also reflects current official materials from Next.js, Expo, and React Native, which support a web-first path with a documented PWA route and a later Expo-based native path if needed.

Appendix D — Functional Requirements Expansion

This appendix expands the concept report into implementation-grade guidance. For the planned platform, concept quality depends on its ability to constrain future decisions, not only to describe ambitions. Accordingly, this appendix turns concept language into practical direction for contracts, interfaces, validation surfaces, governance, or delivery sequencing. The report should therefore be read as both a product-definition document and a quality-control instrument for future design and engineering work.

Appendix E — Data Model Expansion

Appendix F — API Contract Expansion

Appendix G — Validation, Backtesting, and Paper Methodology Expansion

Appendix H — Governance, Guardrails, and Operational Reliability Expansion

Appendix I — UX Architecture Expansion

Appendix J — Component and Design-System Expansion

Appendix K — Development Governance and QA Expansion